

Maze Solver using A* Search

S

Maze Solver using A* Search

Presented by: E SOWMYA

Roll No: 07

Department: BCA

College: K LH

A shortest-path solution for a grid maze

1. Objective

- Represent a maze as a grid of nodes.
- Identify the start position, goal position, open cells, and walls.
- Use the A* Search Algorithm to find the shortest path.
- Use Manhattan Distance as the heuristic.
- Handle cases where the goal is unreachable.
- Visualize the final shortest path.

2. Maze / Grid Representation

- Each cell in the grid is treated as a node.
- 0 represents an open cell that can be visited.
- 1 represents a wall or obstacle.
- Start = (0, 0) and Goal = (4, 5).
- Movement is allowed in four directions: up, down, left, and right.

3. A* Search Algorithm

- A* combines the cost already travelled with an estimate of the remaining distance.
- For every node n , it calculates:
 - $f(n) = g(n) + h(n)$
 - $g(n)$ = actual cost from the start node to n
 - $h(n)$ = estimated cost from n to the goal
 - The node with the lowest $f(n)$ is explored first.
- When the goal is reached, the parent links are used to reconstruct the shortest path.

4. Heuristic – Manhattan Distance

- Manhattan Distance is suitable when movement is limited to four directions.
- Formula:
- $h(n) = |x_{\text{start}} - x_{\text{goal}}| + |y_{\text{start}} - y_{\text{goal}}|$
- It estimates how many horizontal and vertical moves remain.
- The heuristic helps A* search efficiently instead of exploring the entire maze.

5. Python Implementation

```
import heapq

def heuristic(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

def a_star(maze, start, goal):
    open_list = [(0, start)]
    came_from = {}
    g_score = {start: 0}

    while open_list:
        _, current = heapq.heappop(open_list)

        if current == goal:
            path = []

            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            return path[::-1]

        for neighbor in get_neighbors(current, maze):
            new_cost = g_score[current] + 1

            if neighbor not in g_score or new_cost < g_score[neighbor]:
                g_score[neighbor] = new_cost
                priority = new_cost + heuristic(neighbor, goal)
                came_from[neighbor] = current
                heapq.heappush(open_list, (priority, neighbor))

    return None
```

6. Output / Visualization

7

7. Result

- The A* algorithm successfully finds a path from the start to the goal.
- Walls and obstacles are avoided during the search.
- The returned path is reconstructed from the goal back to the start.
- If no valid route exists, the program reports that no path exists.
- The final path is displayed graphically for easy understanding.

8. Conclusion

- The Maze Solver successfully finds the shortest path using the A* Search Algorithm.
- Manhattan Distance provides an effective heuristic for a four-directional grid.
- The project demonstrates how A* can be used for pathfinding and navigation problems.
- The final solution is both computationally useful and visually easy to understand.

Thank You

Questions?